CONTEXT — How This Submission Was Built, End to End

HACKSIMUL8 2026 · PES University · 2 September 2026 A complete record of the working session: what was decided, what broke, how it was diagnosed and fixed, and why the final design looks the way it does.

> **Why this document exists.** The other documents tell you *what* the solution is. This one tells you *how it got there* — including four genuine defects that were found and fixed along the way. If a judge asks "how did you arrive at this?", the answer is here.

Contents

---

1. Timeline

Time	Event
1 Sept, evening	Course material prepared; discovered MATLAB was not installed and the MathWorks account had **no licence linked at all**
2 Sept 08:29	Problem statement PDF received and parsed
08:33 – 08:40	Complete solution package written (11 MATLAB files) — **before MATLAB existed on the machine**
08:45	Licence resolved; R2026a installer downloaded (271 MB)
09:02	Installation complete; smoke test — 9 passed, 0 failed
09:06	**Task 1 and Task 2 first successful run**
09:14	Simulink model built programmatically
09:17	Feedback linearisation verified: 2 μm sag at 17° tilt
09:37	Task 3 data generation launched
10:09	**Task 3 FAILED — 0 of 150 usable samples**
10:41 – 10:55	Dataset re-labelled under a corrected cost function
11:00 – 11:05	Reference paper obtained and read
11:11	Task 2 re-run under the final cost; gains changed 32.84 → 30.18
11:12 – 11:25	Task 3 regenerated against the corrected baseline
11:32	**Task 4 FAILED — mean −168% (worse than no adaptation)**
11:36	Simulink model rebuilt with final gains
11:47 – 11:56	Task 4 re-run with the diagnosis applied
11:56	**Task 4 succeeded (Ki-only): mean +61.2%, TC2 stuck at 0.0%**
~12:30	Model verified against the paper; Euler-Lagrange rotational dynamics fixed to 3.6e-15
~13:00	Task 2 same-structure benchmark added: 1.44x lower ITAE, 28% faster settling than the paper's best, isolated from the feedback-linearisation contribution
13:12	Bound-all-3-gains attempt: +72.2%, TC2 fixed, but TC1 overshoot rose to 37.4%
13:20 - 13:34	Shrinkage tested at 3 strengths - still 17-30% overshoot; ruled out magnitude as the cause
13:35	Asymmetric rule (Kp up-only) tested directly - confirmed
13:39 - 13:47	Adopted, re-run in full: **+72.3% mean, 16.3% max overshoot, TC2 +55.6%**

Time	Event
13:46 - 13:47	120-condition stress test: **+58.1% mean, 90.8% success rate, failure region identified** (moderate payload + wind)
13:47 - 13:56	Part 1 pushed to GitHub; stratified-sampling fix attempted, regenerated, re-tested
13:56	Stratified result: R^2 improved (0.589->0.793) but stress-test performance WORSENED (58.1%->50.1% mean, 9.2%->15.8% failure rate) - rejected, reverted to the pushed Part 1 data/model

Roughly 5.5 hours of working time in total, across two sessions either side of the original 12:00 deadline, of which perhaps 100 minutes was compute.

---

2. Starting position — the night before

The event invitation listed three recommended courses (Simulink Onramp, Control System Design, Stateflow Onramp) and named the toolboxes required.

Two blockers were found immediately:

1. **MATLAB was not installed** — no `C:\Program Files\MATLAB`, nothing on `PATH`, no installer present.
2. **The MathWorks account had no licence linked.** License Center reported "*Your MathWorks Account is not currently linked to any licenses.*"

The second blocker explained several downstream symptoms: the Onramp "Start course" button did nothing (the lessons run in a hosted MATLAB Online session), and every course in the Control System Design path showed as locked.

Course notes were written from the live R2026a course pages plus MathWorks documentation (now in `archive/`). These turned out to matter less than expected — the problem statement was specific enough that the reference paper became the more useful source.

---

3. The problem statement arrives

08:29. A three-page PDF. Text extracted with `pypdf`, but **Table 1 and Figure 1 were embedded images**, so they were extracted separately and read directly.

What Table 1 gave us

Parameter	Symbol	Value
Quadcopter mass	m	0.516 kg
Arm length	l	0.225 m
Gravity	g	9.81 m/s^2
Rotor inertia moment	I_M	3.368×10^{-5} $kg\cdot m^2$
Thrust factor	k	2.996×10^{-6} $N\cdot s^2$
Drag coefficient	b	1.260×10^{-7} $N\cdot m\cdot s^2$
Inertia	Ixx, Iyy	4.984×10^{-3} $kg\cdot m^2$
Inertia	Izz	8.958×10^{-3} $kg\cdot m^2$

What Figure 1 gave us

A **cross ("plus") configuration**: rotor 1 on +x_B, rotor 2 on −y_B, rotor 3 on −x_B, rotor 4 on +y_B, with 1,3 spinning opposite to 2,4. This determined the entire rotor-to-input mapping.

The reading of the four tasks

Task	Literal requirement	What is actually being assessed
1	model in state space	correctness, and whether you can *prove* it
2	PID + "*neat description of the methodology… use the best method available*"	**the tuning method, not the gains**
3	generate data + train an ML model	the dataset design — what are the features
4	test cases with comparisons and merits	honest, quantified comparison

Task 2's wording was read as the highest-value opportunity. It explicitly grades *methodology*. Most teams were expected to tune by trial and present a plot.

Writing code before MATLAB existed

Between 08:33 and 08:40 the complete solution package was written — 11 MATLAB files covering all four tasks — while the installer was still being sorted out. This was a deliberate bet: the physics does not depend on having MATLAB open, and the alternative was idle waiting. It paid off; the first run at 09:06 was 20 minutes after installation completed.

---

4. Task 1 — building and verifying the model

The derivation

Rotor forces. Each rotor produces thrust `f_i = k·ω_i²` along +z_B and reaction torque `τ_i = b·ω_i²` about z_B. Applying **τ = r × F** with the Figure 1 geometry gives the four virtual inputs:

```
U1 = k(ω1² + ω2² + ω3² + ω4²)      total thrust
U2 = k(ω4² − ω2²)                  roll  → τϕ = l·U2
U3 = k(ω3² − ω1²)                  pitch → τθ = l·U3
U4 = b(ω1² + ω3² − ω2² − ω4²)      yaw torque
```

Translational dynamics. Body-frame thrust `[0,0,U1]ᵀ` rotated into the inertial frame by the ZYX Euler matrix, plus gravity.

Rotational dynamics. Euler's equations with a diagonal inertia tensor, plus the gyroscopic term from the spinning rotors.

State vector: 12 states, two per degree of freedom.

Linearised about hover: the system decouples into four independent double integrators.

The verification approach — and why it mattered

Rather than assert the algebra was right, the nonlinear model was **finite-differenced** and compared entry-by-entry against the hand-derived A and B matrices:

```
max|A − A_numerical| = 1.636e-12
max|B − B_numerical| = 3.122e-10
rank(ctrb) = 12 of 12
hover residual ||ẋ|| = 0.000e+00
```

This later proved its worth: when the Task 2 gains changed at 11:11 and the model had to be rebuilt, `verify_all` confirmed in two minutes that nothing had broken.

The key structural finding: all twelve open-loop poles sit at the origin. The quadcopter is marginally stable at best and cannot fly without feedback. This became the motivation for everything downstream.

---

5. Task 2 — the tuning methodology

The central design decision: feedback linearisation

The altitude dynamics are `z̈ = −g + (U1/m)·cos ϕ·cos θ`. Tilt costs lift — at 17° of pitch, 4.47% of it.

The obvious approach is to linearise about hover and treat tilt as a disturbance. **We instead cancel it exactly.** Commanding

```
U1 = m·(g + u_z) / (cos ϕ · cos θ)
```

makes $\ddot{Z} = u_z$ **exactly**, at any tilt angle — algebraically, not approximately.

Verified across a sweep: 0.000034 m sag at 8.6°, 0.000134 m at 17.2°, 0.000298 m at 25.8°. In the Simulink model, **0.000003 m**.

Four tuning methods, presented as a progression

Method	What it optimises	Result
A — analytical pole placement	closed-loop poles, closed form	ITAE 0.5311
B — `pidtune`	a linear approximation	ITAE 11.1617
C — `pidtune` at a target crossover	same, faster	ITAE 5.0895
D — ITAE optimisation on the nonlinear model	**what actually flies**	**ITAE 0.0840**

Method D is the answer to "*use the best method available*": A, B and C all optimise an approximation, while D optimises the real system including saturation, anti-windup and the 100 Hz discrete controller.

A trap found in Method A

The first version designed Kp and Kd as a PD ($\zeta = 0.9$) and then added Ki afterwards. That produced **20% overshoot with no settling** — integral action moves the closed-loop poles, destroying the damping just designed for.

Fix: place all three poles simultaneously via $K_d = 2\zeta\omega_n + p$, $K_p = \omega_n^2 + 2\zeta\omega_n p$, $K_i = \omega_n^2 p$.

Method A still overshoots 26%, and the explanation is instructive: **pole placement controls poles, but a PID also introduces a zero** at $s = -K_i/K_p$. A slow zero causes overshoot regardless of pole locations. Method D avoids this because it optimises the measured response, in which zeros are automatically accounted for.

An observation that generated a testable prediction

Method D drove **Ki to exactly zero**. The reason: feedback linearisation cancels gravity via feed-forward, so in the nominal case there is no steady-state error for integral action to remove — it would only add overshoot.

The consequence was noted at the time: with Ki = 0 the controller is *not robust* to an unknown payload. This produced a prediction, written down before any Task 3 data existed:

> *The ML model should learn to reintroduce integral action when it detects a payload.*

Section 7 records how that prediction was confirmed.

---

6. Reading the reference paper — and what it changed

11:00. The paper — Mien & Tu (2024), IJRCS 4(4) — sits behind a Cloudflare bot check. The user cleared it manually and saved the PDF; it was then parsed locally.

Four findings, each of which changed something

(a) Their Table 1 is identical to ours. All eight parameters match. Confirms our inputs.

(b) Their Eq. (6) includes air-resistance terms `Ax, Ay, Az` that Table 1 never quantifies — in their paper *or* in our problem statement.

This was the important one. Our Task 2 documentation had been claiming, flatly, that "*Ziegler–Nichols is undefined for this plant*". The paper **applies ZN successfully** and publishes measured values (`kth = 124.99` , `τth = 3.52 s`).

The resolution: with `Az ≠ 0` the altitude plant is `1/(s(ms+Az))` — one pole off the origin, so a finite ultimate gain exists. With `Az = 0`, which is the only consistent reading of Table 1, it collapses to a pure double integrator and ZN becomes degenerate.

The claim was rewritten in both the code and the documentation to state the qualification explicitly. An unqualified version would have been caught by any judge who had read the reference.

(c) Their tuning formulas were extracted and verified. Applying their Eq. (24) and Eq. (25) to their published `kth` and `τth` reproduces their published Table 2/3 altitude gains to **under 4×10^{-5}**:

Method	Their published	Our computed
Ziegler–Nichols	74.9940, 42.6102, 32.9974	identical
Tyreus–Luyben	56.8136, 7.3365, 31.7435	identical

That verification was done *before* any comparison, to confirm the paper had been read correctly.

(d) Their rotational dynamics use the Euler-Lagrange form, Eq. (11) — `η¨= J(η)⁻¹[τ_B − C(η,η)η]` with a configuration-dependent inertia matrix — not the simplified body-frame Euler equations we had originally implemented.

We measured the divergence: identical at hover, but **22% apart at 17° of tilt** and 47% at 40°. Since the task is to reproduce the paper's model, `quad_dynamics.m` was rewritten to implement Eq. (8), (11) and (12) verbatim. Agreement is now **3.6e-15 at every tilt angle**, verified against an independent reference implementation built separately from the paper text.

Critically, this changed **nothing downstream**: at hover `J(0) = diag(Ixx,Iyy,Izz)` and `C(0,0) = 0`, so the linearised A and B matrices are identical, and the Task 2 gains came back bit-for-bit unchanged (delta = 0.000e+00). The simplified form remains available as `P.rot_model = 'euler'` for comparison.

(e) Their altitude control law, Eq. (27), is a plain PID — no gravity feed-forward, no tilt compensation. This made our feedback linearisation a citable differentiator rather than just a design preference.

The benchmark this produced

All four designs simulated on the identical plant, each in the configuration its author intended, stepping to the paper's own setpoint of 2.0 m:

Design	Ts [s]	OS [%]	ITAE	Tilt sag [m]
Ziegler–Nichols (Mien & Tu)	4.610	12.13	1.7259	0.004522
Tyreus–Luyben (Mien & Tu)	10.300	4.96	6.6841	0.034002
MATLAB PID Tuner (Mien & Tu)	1.490	0.00	0.3319	0.008033
Ours	**0.980**	**0.00**	**0.1874**	**0.000134**

Cross-check: their paper reports Tyreus–Luyben achieving "steady-state error of less than 1%". Our simulation of their gains gives **0.592%** — inside their stated bound, which independently confirms the reproduction.

---

7. Task 3 — four defects and their fixes

This was the hardest part of the session. Four separate defects, each found by a result that did not make sense.

Defect 1 — the controller could see the answer

Symptom (10:09): after a 32-minute run, `Usable samples: 0 of 150`.

Diagnosis: `sim_altitude` used a single mass `P.m` for **both** the plant and the controller's feedback linearisation. So "adding a payload" also told the controller about it, and the feed-forward compensated perfectly. Payload had no effect on the response, feature 6 (thrust ratio) read exactly 1.000 every time, and there was nothing to learn. The optimiser then wandered to nonsense gains, and the sanity filter correctly rejected all 150.

Fix: split into `m_ctrl` — the mass the *controller assumes* — and the plant's true mass.

Verification: immediately after, a ×1.8 payload produced 21% steady-state error and feature 6 read **1.736 ≈ the true 1.8**. The signal existed.

> This is the single most important fix in the project. Without it there is no Task 3.

Defect 2 — the optimiser brute-forced instead of integrating

Symptom: with mass unknown, `fminsearch` returned **Kp ≈ 1978**.

Diagnosis: it discovered it could mask a steady-state error with enormous proportional gain rather than using integral action. Such gains are physically useless — they amplify sensor noise and sit permanently in saturation.

Fix: added a gain-regularisation term `2e-4·(Kp² + Kd²)` to the cost, expressing the real engineering constraint rather than an arbitrary cap.

Defect 3 — `fminsearch` never explored Ki

Symptom: even after Defect 2 was fixed, Ki came back at ≈ 0 for every condition, and steady-state error persisted.

Diagnosis: `fminsearch` builds its initial simplex by perturbing each component of the starting point. A component starting at **exactly zero** gets an almost invisible initial step, so integral action was never meaningfully explored.

Fix: seed Ki at 5.0 rather than 0.

Result — this is the project's headline finding:

Payload	Optimal Ki	Final altitude
×1.0	**0.00**	1.0171
×1.4	5.50	1.0000
×1.8	11.01	0.9999
×2.2	**16.05**	1.0000

Ki is exactly zero at nominal and rises almost linearly with payload excess — precisely the prediction made in §5 before any data existed. Steady-state error goes from 36 cm to zero; ITAE improves up to 31×.

> A numerical artefact had been hiding the best result in the project.

Defect 4 — overshoot was under-weighted

Symptom: the first Task 4 run showed the learned controller flying to **1.62 m on a 1.00 m command**.

Initial diagnosis was wrong. The cost function's overshoot weight (0.05) was blamed, and the dataset was re-labelled at 0.30. Measuring the effect showed the labels were **never the problem** — optimal controllers in the training set averaged 1.0% overshoot, max 7.1%.

The relabelling helped anyway, for a different reason: warm-starting plus a second seed per condition produced better-converged, more self-consistent optima. Cleaner labels are more learnable, and mean held-out R^2 rose from 0.395 to 0.589.

The real cause of the 1.62 m overshoot was the *model's prediction*, not the training targets — which §8 addresses.

A consistency issue found late

At 11:11, re-running Task 2 under the corrected cost changed the gains from `Kp = 32.8366` to `Kp = 30.1847` — an 8% shift.

That mattered because Task 3's training data used the old baseline for its identification flights. Rather than argue 8% was negligible, **Task 3 was regenerated from scratch** against the corrected baseline (13 minutes), and the Simulink model was rebuilt so that code, model and documentation all carry identical gains.

---

8. Task 4 — the failure that produced the key insight

The failure

11:32. With the model setting all three gains:

```
TC1  predicted Kp = 0.050   ITAE 152.046 vs fixed 18.762   -710.4%   overshoot 353%
TC4  predicted Kp = 0.050   ITAE  31.074 vs fixed 14.855   -109.2%
TC5  predicted Kp = 0.050   ITAE  29.577 vs fixed 12.173   -143.0%
MEAN: -168.3%
```

The ML controller was **substantially worse than doing nothing**.

The diagnosis

`0.050` is the safety clamp's floor. The model was predicting **negative Kp** on test conditions outside the training grid — catastrophic extrapolation — and the clamp caught the sign but not the magnitude. With essentially no proportional action, the controller barely responded.

The fix was indicated by the Task 3 regression accuracy, which had been reported honestly all along:

Gain	R^2	Interpretation
Ki	0.70 – 0.87	uniquely determined — a given steady-state error needs a specific integral strength
Kp	0.21 – 0.52	many (Kp,Kd) pairs give near-identical cost
Kd	0.14 – 0.37	same — the cost surface has a flat valley

Kp and Kd labels are intrinsically noisy. No model can predict noise.

And the oracle confirmed adapting them was unnecessary: for TC1 it wanted Kp = 20.2 against our baseline 30.2 (close), while Ki went 0 → 12.2. **Ki is where the entire benefit lies.**

The fix

Adapt only Ki. Hold Kp and Kd at their Task 2 values. Bound the prediction to the range observed in the training labels, so the model interpolates but never extrapolates.

```
BEFORE:  mean -168.3%
AFTER:   mean  +61.2%
```

Test case	Improvement	Predicted Ki	Oracle Ki
TC1 heavy payload ×1.8	**+80.7%**	11.43	12.23
TC2 strong wind + gust	−0.0%	0.00	19.52
TC3 large step + tilt	**+73.4%**	2.36	3.11
TC4 combined worst case	**+70.2%**	8.03	8.39
TC5 noise + payload	**+81.7%**	6.84	8.39

On four of five, predicted Ki lands within ~20% of the oracle's — from three seconds of observation, with no knowledge of the payload.

> **The fix was methodological, not numerical.** It was not hyperparameter tuning. It was asking *which parameters does my data actually identify?*, reading the answer off the R^2 values, and restricting the model accordingly.

The one unresolved case

TC2 shows exactly 0.0% because the model predicted Ki = 0 — it declined to adapt.

This is explainable rather than mysterious. TC2 is the *lightest* payload case (×1.1), so the thrust-ratio feature read near-nominal and the model correctly inferred no payload correction was needed. What actually hurts TC2 is **wind**, and the feature set identifies mass well but wind poorly.

The named next step: add a wind-discriminating feature — most plausibly the low-frequency component of the thrust residual after removing the mass estimate.

This is left as a documented limitation rather than patched, because the model declining to act when it cannot identify a condition is *better* behaviour than guessing confidently and getting it wrong — which is exactly what the previous version did.

(This was the state of Task 4 as of the first "final" push, around 12:00. The rest of this section records what happened after — the user asked whether the model actually matched the reference paper, which reopened Task 1, and then asked directly whether Task 4 could be improved further, which reopened the section above.)

8a. Verifying the model against the paper, properly (post-12:00)

User's question, verbatim: "*Does the dynamic mathematical model of 6 degree uav match the paper correctly? like perfectly or not? tell me that too? if the model is inaccurate then fitting an ml model for that system wont be meaningful, so do that check.*"

This was the right question to ask, and the honest answer required actually building a second, independent implementation of the paper's equations and comparing outputs numerically — not eyeballing the algebra.

What the check found: translational dynamics (their Eq. 7) matched to 3.6e-15. But the **rotational** dynamics did not — the model had been using simplified body-frame Euler equations, while the paper uses the full Euler–Lagrange form (their Eq. 8/11/12) with a configuration-dependent inertia matrix. The two are identical at hover and diverge with tilt: **22% apart at 17°, 47% apart at 40°.**

The user's response was direct: "*then fix that part because we have to match it exactly … we want task 1,2,3 to match whatever is in the paper, task 4 is the later part which we have to make it better.*" That sentence set the standing rule for the rest of the session: **Tasks 1–3 reproduce the paper exactly; Task 4 is where we are allowed — expected — to exceed it.**

`quad_dynamics.m` was rewritten to implement Eq. (8), (11) and (12) verbatim, verified against an independently built reference implementation. Agreement is now **3.6e-15 at every tilt angle tested, up to 40°.** Nothing downstream changed — Tasks 2–4 integrate only the altitude equation, which was already exact, and the linearised A/B matrices are identical at hover regardless of which rotational form is used (`J(0) = diag(Ixx,Iyy,Izz)` , `C(0,0) = 0`).

The user then asked a clarifying question that exposed a real communication gap: "*so what and how much are we done with currently? … there are no numbers to match from the research paper.*" This was correct — Task 1 has no results table to compare against, only equations. "Matching the paper" for Task 1 means the

algebra agrees when fed identical inputs, which is a different kind of claim than Task 2's "our numbers beat their numbers." Both are true, but conflating them was sloppy, and it was worth a full explanation distinguishing the two before continuing.

8b. Is Task 2 actually better, or just different? (post-12:00)

User's question: "*wdym what shd we do in task2? are u sure we dont need to match it?*" — followed by, once shown the answer, "*is ours better than the research paper then? how is it better than it? tell me properly.*"

The Task 2 comparison up to that point mixed two separate things: a better **tuning method** and a better **controller structure** (feedback linearisation). A judge could reasonably ask whether the improvement was really about tuning, or just about using a fundamentally different (and easier) controller. The honest way to answer was to isolate them: re-tune with our ITAE method but **without** feedback linearisation — i.e. exactly the paper's plain PID structure — and compare against their three published designs on identical terms.

Design (all plain PID)	Ts[s]	OS[%]	ITAE
Ziegler-Nichols (theirs)	4.610	12.13	1.7259
Tyreus-Luyben (theirs)	10.300	4.96	6.6841
MATLAB PID Tuner (theirs)	1.490	0.00	0.3319
Ours, same structure	1.080	0.03	0.2307

On an identical structure: 1.44× lower ITAE, 28% faster settling, gains 2.7× smaller. The improvement decomposes cleanly: 0.3319 (their best) → 0.2307 (our tuning alone) → 0.1874 (tuning plus feedback linearisation). Both `TASK2_final.md` and `task2b_zn_tyreus_luyben.m` were updated with this decomposition, including the fairness caveat that ITAE is partly circular (we optimise it, then report it) while settling time is the independent check.

8c. Reopening Task 4 — "is that really all we can do?"

With ~2 hours still available (user: "*we still have 2 more hours*", then "*But after u r done running this, what all things are we left with*", then, pointedly: "*can we improve it more and better?*"), Task 4's Ki-only result (+61.2%, TC2 stuck at 0.0%) was revisited rather than left as final.

Attempt: bound all three gains to the training range, instead of freezing two. Non-destructive test first (`task4b_bounded_all3.m`, kept isolated from the shipped script until proven). Result: mean rose to **+72.2%**, and TC2 specifically went from 0.0% to **+55.7%** — its own Kp prediction had been correctly detecting the disturbance all along; freezing Kp had simply discarded that signal.

Folded into the real script and re-run in full (with the oracle comparison and figure). The numbers held — but a new problem appeared: **TC1 overshoot jumped from ~0% to 37.4%.** Letting Kp fall to the training floor (15.83 against a baseline of 30.18) had removed damping that the model's own elevated Ki prediction needed.

The user asked, reasonably: "*are u sure there is no other alternative/method?*" This prompted a further, more careful test rather than accepting the trade-off. **Shrinkage toward baseline** (blending the prediction with baseline at 90%, 60% and 50% strength) was tried next — and *still* left 17–30% overshoot on TC1 at every strength tested. That result ruled out "how far the gain moves" as the cause and pointed at something directional instead.

Hypothesis, tested directly: Kp *increasing* (what TC2 needed) was never a problem in any case; Kp *decreasing*, once Ki had risen, was dangerous regardless of magnitude. Testing an asymmetric rule — Kp may rise above baseline but never fall below it, Ki free within the training range, Kd held at baseline — confirmed it:

```
Method                          Mean     Max overshoot   TC2
Freeze Kp, Kd                  +61.2%    ~0%             0.0% (broken)
Bound all 3 symmetrically      +72.2%    37.4%           +55.7%
Asymmetric (adopted)           +72.3%    16.3%           +55.6%
```

The asymmetric rule does not trade one thing for another — it beats both prior attempts on every measured axis simultaneously, because it is built on a tested reason for the failure rather than a blanket constraint.

User, immediately after: "*try testing the dataset for around a lot of values to understand where we stand.*" Five curated cases cannot establish whether a method generalises. `task4_stress_test.m` was written to draw 120 conditions at random from the full envelope (a seed disjoint from both the Task 3 training draw and the Task 4 curated cases) and compare fixed PID against the self-tuner on every one, without the (expensive) oracle step.

The honest result:

```
Mean improvement       : +58.1%   (median +73.6% - a real cluster of bad cases pulls
                                     the mean below the median, not evenly spread noise)
Cases worse than doing
  nothing              : 11 / 120 (9.2%)
Max overshoot          : 50.86%  (worse than any curated case showed)
```

The five worst conditions were, without exception, **moderate payload (1.1×–1.4×) combined with meaningful wind** — exactly the region already known to be sparse in training (10 of 150 samples). Correlation across all 120 conditions confirmed it statistically: `corr(improvement, mass) = +0.563`, `corr(improvement, wind) = −0.310`. The model reads payload well and wind poorly — previously an anecdote from one test case (TC2), now a quantified, general pattern.

This is reported as the real result, not softened. The curated 5-case table understates the risk; the stress test is what surfaced it. `TASK4_final.md` was rewritten around both sets of numbers together, with the failure region named and the correlation evidence shown, because a precisely characterised weakness is more credible to present than a vague one — and considerably more credible than one left undiscovered.

8d. The obvious fix, tried and rejected

With the failure region identified precisely, the direct fix seemed clear: regenerate the Task 3 dataset with **stratified sampling** - explicitly pack part of the training budget into the confirmed weak region (moderate mass, meaningful wind) instead of relying on random draws to find it. Implemented as ~35% of the 150-sample budget laid on a grid inside that region, the rest left as broad random coverage as before. Retrained and re-run through the full chain: Task 3 -> model selection -> curated Task 4 -> the 120-condition stress test.

The model's own regression accuracy improved substantially - mean held-out R^2 rose from 0.589 to 0.793, with Kp specifically going from ~0.47 to 0.800 and Kd from ~0.22 to 0.700. By the metric used to select model classes in Task 3, this was a clear win.

But the stress test - the same 120-condition sweep, re-run fresh - showed real performance had gotten WORSE, not better:

```
                          i.i.d.     stratified
mean improvement          +58.1%     +50.1%
median                    +73.6%     +68.0%
cases worse than baseline   9.2%      15.8%
max overshoot               50.9%     80.0%
corr(improvement, wind)   -0.310     -0.323   (barely moved - the thing being fixed)
```

The diagnosis: packing over a third of the training budget into one region reshaped what the model implicitly treats as "normal." It became measurably better at predicting gains for data drawn from that reshaped distribution (which is what the R^2 numbers measure) but more aggressive - and more overshoot-prone - across the much larger set of ordinary conditions the stress test actually samples from. Better test accuracy on a redesigned distribution did not transfer to the distribution that matters, and the specific correlation the fix targeted (wind sensitivity) was essentially unchanged.

Action taken: the data and model files were reverted to the pre-stratification version (recovered from the git commit already pushed for Part 1, which had been proven better by the same stress test) rather than shipping the version with higher R^2 but worse real performance. `STRATIFY` in `task3_generate_data_train_ml.m` defaults to `false`; the stratified code path is left in place, documented, and reproducible, but is a recorded negative result rather than a recommendation. This is treated as a genuine finding worth keeping, not a mistake worth hiding: it is direct evidence that a regression metric (R^2 on a chosen split) and the metric that actually matters (closed-loop performance on the real distribution) can move in opposite directions, and that only measuring the latter would have caught it before shipping.

---

9. Key decisions and the reasoning behind them

Decision	Alternative	Why
Feedback-linearise the altitude channel	linearise about hover (what the paper does)	exact at any tilt; measured 253× less sag
Adapt only Ki	adapt all three gains	Kp/Kd labels are intrinsically noisy; adapting them gave −168%
Infer mass from thrust ratio	feed mass in as a model input	in flight you do not know the payload; the latter is gain scheduling, not self-tuning
Compare four model classes on held-out data	assume a neural network	the deepest network scored *worse than linear*; selection caught it
Include an oracle in Task 4	compare only against the fixed PID	"we beat the baseline" only proves the baseline was weak
Set `Ax = Ay = Az = 0`	invent plausible drag values	Table 1 gives no values — zero is the only consistent reading
Regenerate Task 3 after the 8% gain change	argue 8% was negligible	slides must match what the code produces live
Bound predictions to the training range	rely on a fixed safety clamp	the clamp caught the sign but not the magnitude

10. What is solid and what is provisional

Solid — measurements that hold regardless

- Table 1 parameters and the derived quantities
- `max|A - A_numerical| = 1.636e-12`; rank(ctrb) = 12; all poles at origin
- Final gains `Kp = 30.1847, Ki = 0, Kd = 10.3994`; Ts = 0.98 s; OS = 0.00%
- Tilt sag sweep: 0.000034 / 0.000134 / 0.000298 m at 8.6° / 17.2° / 25.8°
- The benchmark against Mien & Tu, including the 0.592% cross-check
- Ki versus payload: 0 → 5.50 → 11.01 → 16.05
- Correlations: thrust ratio vs Ki r = +0.840; mass ratio vs Ki r = +0.785
- Task 4 test-case results: mean +61.2%

Provisional — would change if the problem framing changed

- The selected model class and its exact R^2 values (depend on the current label set)
- The cost-function weights in `alt_cost.m`
- The decision to adapt only Ki (correct for *this* feature set; a wind feature could change it)
- The sampled operating envelope

If `alt_cost.m` or the Task 2 gains change, Tasks 3 and 4 must both be re-run. This happened once during the session and is the reason the dependency chain is documented in `RUNBOOK.md`.

11. Lessons that generalise

1. A result that is too clean is a bug report. `0 of 150 usable samples` was obviously broken. But before that, the controller compensating *perfectly* for every payload looked like success — it was actually the controller being handed the answer.

2. Report what your data can and cannot identify. The R^2 split (Ki 0.70+, Kp/Kd ~0.2–0.5) was visible from the first training run. Acting on it — rather than treating it as a number to improve — was what turned −168% into +61%.

3. Verify against the source before comparing to it. Reproducing the paper's published gains to 4×10^{-5} came *before* any performance comparison. Without that step, a discrepancy could mean either "we are better" or "we misread their method".

4. Consistency beats marginal accuracy. The 8% gain change was arguably negligible. But slides that disagree with a live demo are worse than slightly suboptimal gains.

5. Numerical artefacts hide real physics. `fminsearch` not exploring a component that starts at zero is a mundane implementation detail. It was concealing the best result in the project.

Appendix — what to read next

Document	Contents
`RUNBOOK.md`	How to run everything; every figure explained — which script made it and how
`TASK1_final.md`	Model derivation, state space, verification, presentation script, Q&A
`TASK2_final.md`	Feedback linearisation, four tuning methods, paper benchmark, Q&A
`TASK3_final.md`	Dataset design, model selection, defects, Q&A
`TASK4_final.md`	Self-tuner, five test cases, why only Ki is adapted, Q&A
`README_HANDOFF.md`	Index and file guide

Repository: https://github.com/VismayHS/H8